



JOHNS HOPKINS

WHITING SCHOOL
of ENGINEERING

FPGA Design Using VHDL

Module 3F: Lab 3 Hints and Pulse Generators

Designing a Pulse Generator

- We want a component that can be given a desired count value and generates a pulse after that many number of clock cycles
 - Our pulse generator will count from 0, so we need to keep that in mind!
- This allows reusability of the component for multiple applications that might require a pulse generator
- Output pulse should be 1 clock width once count is reached and reset to 0 at that time or when reset is triggered
 - This allows the pulse to be used to trigger a single event

Pulse Generator I/O



- Inputs:
 - clk: System Clock (100Mhz)
 - reset: System Active High Reset
 - maxCount: number of clock pulses to count
 - Note: This could be a generic if we don't have a changing max count
- Outputs:
 - pulseOut: Output Pulse, 1 clock width wide

Pulse Generator Entity

- From the given inputs and outputs, we can work out this entity:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity pulseGenerator is
    Port ( clk : in STD_LOGIC;
           reset : in STD_LOGIC;
           maxCount : in unsigned(26 downto 0);
           pulseOut : out STD_LOGIC);
end pulseGenerator;
```

Pulse Generator Architecture

- An example architecture:

```
architecture Behavioral of pulseGenerator is
    signal pulseCnt : unsigned(26 downto 0);
    signal clear : std_logic;
begin
    --Pulse Generator
    process(clk, reset)
    begin
        if(reset = '1') then
            pulseCnt <= (others=>'0');
        elsif(rising_edge(clk)) then
            if (clear = '1') then
                pulseCnt <= (others=>'0');
            else
                pulseCnt <= pulseCnt + 1;
            end if;
        end if;
    end process;

    clear <= '1' when (pulseCnt = maxCount) else '0';
    pulseOut <= clear;
end Behavioral;
```

Pulse Generator Testbench

```
library IEEE;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
use work.all;

-- empty entity
entity testPulseGenerator_tb is
end testPulseGenerator_tb;

architecture testbench of testPulseGenerator_tb is

    signal clock : std_logic;
    signal reset : std_logic;
    signal MaxCounter1 : unsigned(16 downto 0);
    signal Pulse1KHz : std_logic;

begin

    pulseGenerator_cut: entity pulseGenerator port map (
        clk => clock,
        reset => reset,
        MaxCounter => MaxCounter1,
        Pulse => Pulse1KHz);

    -- referenced from the start of the simulation
    reset <= '0', '1' after 20 ns, '0' after 100 ns;
    -- Binary value for 100,000
    MaxCounter1 <= "11000011010100000";

    process -- no sensitivity list
    begin
        clock <= '1';
        wait for 5 ns;
        clock <= '0';
        wait for 5 ns;
    end process;

end testbench;
```

Pulses Generated using Counters

- Why do I need a pulse?
- Enable signal for sequential designs
- **DO NOT USE ENABLE AS A CLOCK**
 - **DO NOT USE MULTIPLE CLOCKS**

```
process(pulse, reset)
begin
    if(reset = '1') then
        cntr3bit <= (others => '0');
    elsif(rising_edge(pulse)) then
        cntr3bit <= cntr3bit + 1;
    end if;
end process;
```

```
process(clk100, reset)
begin
    if(reset = '1') then
        cntr3bit <= (others => '0');
    elsif(rising_edge(clk100)) then
        if(pulse = '1') then
            cntr3bit <= cntr3bit + 1;
        end if;
    end if;
end process;
```

VHDL Coding Styles

-- DON'T DO THIS

```
process(pulse, reset)
begin
    if(reset = '1') then
        cntr3bit <= (others => '0');
        elsif(rising_edge(pulse)) then
            cntr3bit <= cntr3bit + 1;
        end if;
    end process;
```

-- This is fine

```
process(clk100, reset)
begin
    if(reset = '1') then
        cntr3bit <= (others => '0');
        elsif(rising_edge(clk100)) then
            if(pulse = '1') then
                cntr3bit <= cntr3bit + 1;
            end if;
        end if;
    end process;
```

-- Please don't do this

```
process(clk100, reset)
begin
    if(reset = '1') then
        cntr3bit <= (others => '0');
        elsif(rising_edge(clk100) and pulse = '1') then
            cntr3bit <= cntr3bit + 1;
        end if;
    end process;
```


Seven-Seg Controller



- Each character is a `std_logic_vector(3:0)`
- For this component, re-use the pulse generator to make a 1kHz pulse which will be used to increment a counter
- Use the 1kHz counter to determine which character to display by setting the anodes and cathodes to the corresponding digit

Design Flow



- For Lab 3, the recommended flow would be:
 - Design the seven-segment controller and test it using constant characters
 - Once verified, test the 1Hz shift register by inputting constant characters
 - At the end, tie in the slider switches to complete the assignment